

# SDD-Crash-Course: Spec-Driven Development in Practice

Mastering Context-Driven Development (Conductor), Delta Contracts (OpenSpec), and Spec-as-SSOT (SPEC.md)

## 1. Conductor (Context & TDD Engine)

**Unit of Work:** Track (`conductor/tracks/<id>/`)

- Persistent repo constitution (`product.md`, `tech-stack.md`, `workflow.md`).
- Interactive 7-Section Track `spec.md` & phased `plan.md`.
- Strict **Red** → **Green** → **Refactor** TDD execution (`/conductor:implement`) with `git notes` & SHA checkpoints.

## 2. OpenSpec (Delta Contract Engine)

**Unit of Work:** Capability Delta

(`openspec/changes/<id>/`)

- Two-Plane isolation: `openspec/specs/` (Current Truth) vs. `openspec/changes/` (Proposed Delta).
- Normative RFC 2119 (**SHALL**, **MUST**) + 4-hashtag **#### Scenario:** (**GIVEN**/**WHEN**/**THEN**).
- Deterministic merge via `/opsx:verify` & `/opsx:archive`.

## 3. SPEC.md (Single Source of Truth)

**Unit of Work:** Whole-System Blueprint (`SPEC.md`)

- Canonical 8-Section architecture, schema, API, and deployment specification.
- Separates **Essential Domain Intent** from incidental implementation details.
- Verified by the **Clean-Room Rebuild Test** against read-only adversarial tests.

## What You Will Build & Verify in This Crash Course (100% Executable Python Playground)

- **6 Self-Diagnosing Hands-On Labs** (`labs/lab_01` to `lab_06`): Rate Limiter, Brownfield Lease Manager with Fencing Tokens, Circuit Breaker State Machine, Token Bucket Drift & Scorecard Refactoring, KVCache SDD Code Review & Stacked PRs, and CoursePulse Clean-Room Rebuild.
- **One-Command Verification Harness:** Run `./self_diagnose_all.sh` to execute all **6 lab graders** and **37 traceable unit tests** (`test_req0001_...`) in < 1 second.

# Why Conversational “Vibe Coding” Fails on Real Engineering Repos

Replacing ephemeral chat prompts with version-controlled specifications, TDD plans, and adversarial gates

## Anti-Pattern: The Vibe Coding Amnesia Loop

- 1. Ephemeral Intent:** Requirements live in transient chat history and vanish when the session resets.
- 2. Silent Brownfield Drift:** The agent guesses existing conventions, introduces unapproved dependencies, or mutates FIFO caches into LRU without warning.
- 3. Sycophantic Self-Testing:** When the same unconstrained prompt writes both code and tests, the agent weakens assertions so broken code passes.
- 4. Unreviewable PR Slop:** 2,000-line diffs with zero traceability to business requirements.

## Solution: The 4-Layer SDD Engineering Loop

**Layer 1 — Repository Constitution:** `conductor/product.md`, `tech-stack.md`, `workflow.md`, `openspec/config.yaml`.

**Layer 2 — Behavioral Contracts:** Numbered `REQ-XXXX` invariants with RFC 2119 **MUST/SHALL** and **GIVEN/WHEN/THEN** scenarios.

**Layer 3 — Sequenced TDD Plan:** Small, reviewable tasks (`plan.md` / `tasks.md`) decomposed into Stacked PR batches.

**Layer 4 — Verified Execution:** Traceable `test_reqXXXX_*` unit tests, `git notes`, and `/opsx:archive` sync.

## Information-Theoretic Principle of Spec-Driven Development

A specification  $S$  and repository constitution  $K$  drive conditional behavioral entropy  $H(\text{Behavior}(C) \mid S, K) \rightarrow 0$  across every observable contract (inputs, state transitions, error codes, latency bounds), while leaving incidental implementation choices constrained by styleguides. **Source code becomes a deterministic compilation target of the specification.**

# The 3 Levels of Spec-Driven Development Maturity

Moving from disposable pre-coding specs to living delta synchronization and deterministic clean-room rebuilds

## Level 1: Spec-First

“Write a spec before coding, then discard it.”

- Engineers draft a design doc or prompt spec to generate v1 of a feature.
- Subsequent bugfixes and hotfixes edit source code directly.
- **Failure Mode:** Within 2 sprints, the spec is stale documentation that misleads future agents.

## Level 2: Spec-Anchored

“Living specs synchronized on every PR.”

- Powered by **OpenSpec** (`openspec/specs/`) and **Conductor** (`conductor/tracks/`).
- Every change proposes explicit **ADDED** / **MODIFIED** / **REMOVED** / **RENAMED** requirement deltas.
- **Guarantee:** CI and `/opsx:verify` block PRs whenever code and specs diverge.

## Level 3: Spec-as-Source (SSOT)

“The spec can regenerate the system from scratch.”

- Powered by the canonical **8-Section SPEC.md** blueprint.
- Zero raw customer secrets or hardcoded environment IDs.
- **Guarantee:** Passes the **Clean-Room Rebuild Test** in an empty `/tmp/` directory against locked adversarial tests.

## Universal Decision Rule: Which Level & Tool Do I Pick Today?

- **Single-repo feature, bugfix, or TDD refactor?** Use **Conductor** (`/conductor:newTrack` → `/conductor:implement`).
- **Cross-cutting capability contract or brown-field delta?** Layer **OpenSpec** (`/opsx:propose` → `/opsx:archive`).
- **Re-architecting or extracting a portable microservice blueprint?** Author an **8-Section SPEC.md** and run the Rebuild Test.

# Conductor: Context-Driven Development & TDD Track Execution

“Measure Twice, Code Once” — Persistent repository context, interactive specifications, and Git-backed checkpoints

## Step 1: `/conductor:setup`

### One-Time Repo Constitution:

- `conductor/product.md` (vision & personas)
- `conductor/product-guidelines.md`
- `conductor/tech-stack.md` (approved stack)
- `conductor/workflow.md` (TDD & coverage > 80%)
- `conductor/tracks.md` (master track registry)

## Step 2: `/conductor:newTrack`

### Interactive Spec & Phased Plan:

- Asks 3–5 targeted clarifying questions (Additive vs. Exclusive).
- Writes `conductor/tracks/<id>/spec.md`.
- Writes hierarchical `plan.md` with `[ ]` tasks and Phase Manual Verification gates.
- Sandboxed in Plan Mode (`^conductor/`).

## Step 3: `/conductor:implement`

### Autonomous Red-Green-Refactor TDD:

- Marks task `[~]` → writes failing test (**Red**).
- Writes minimal code (**Green**) & refactors.
- Commits code, attaches `git notes`, records `[x] <7-char-sha>`, and runs Phase Checkpoint (`[checkpoint: <sha>]`).

## Day-2 Operations in Conductor: Auditing, Principal Review & Logical Git Revert

- `/conductor:status`: Audits `tracks.md` and active `plan.md` files to report completed `[x]`, in-progress `[~]`, and blocked tasks.
- `/conductor:review`: Audits diffs against `spec.md`, `tech-stack.md`, and styleguides; appends `## Phase: Review Fixes` to `plan.md`.
- `/conductor:revert`: Reconciles implementation SHAs, `conductor(plan)` SHAs, and `conductor(checkpoint)` SHAs to cleanly roll back an entire Task, Phase, or Track.

# OpenSpec: Two-Plane Architecture & Deterministic Delta Algebra

Isolating current system truth (`openspec/specs/`) from active work-in-progress proposals (`openspec/changes/`)

## Plane 1: `openspec/specs/<capability>/spec.md`

### The Living System of Record (Current Truth):

- Organized by architectural capability (`rate-limiter`, `lease-manager`, `enrollment`), never by sprint ticket.
- Every requirement has a stable ID (`REQ-0001`) and RFC 2119 normative verb (`SHALL`, `MUST`, `MUST NOT`).
- Every requirement is paired with 4-hashtag `#### Scenario:` blocks using `GIVEN / WHEN / THEN`.

## Plane 2: `openspec/changes/<change-id>/`

### Isolated Delta Bundle (`proposal.md`, `design.md`, `tasks.md`, `specs/`):

- Expresses exact mutations via 4 delta headers: `ADDED`, `MODIFIED`, `REMOVED` (requires **Reason & Migration**), and `RENAMED`.
- **Deterministic 4-Step Archive Merge** (`/opsx:archive`):  
1. `Pre-Validate` → 2. `RENAMED` → 3. `REMOVED` → 4. `MODIFIED` → 5. `ADDED`.

## The 99% / 1% OpenSpec Rule & The “Core 4” Daily Loop

- **1% Terminal CLI:** Run `openspec init` once per repo (and `openspec validate -all -strict -json` in CI).
- **99% AI Slash Commands:** Drive all daily work through `/opsx:explore` (read-only investigation) → `/opsx:propose` (generate `proposal.md`, `specs/`, `design.md`, `tasks.md`) → `/opsx:apply` (implement tasks) → `/opsx:verify` & `/opsx:archive` (audit & merge deltas).

# Wiring Conductor + OpenSpec + Dedicated Spec Skills Together

How `conductor/workflow.md` delegates specification quality to `writing-arch-specs` and OpenSpec contracts

## Dual-Engine Repository Wiring

### 1. Dedicated Spec Skill:

`.agents/skills/writing-arch-specs/SKILL.md` defines the 7-Section Track Spec Anatomy, RFC 2119 rules, and ADR tables.

### 2. Workflow Hook (`conductor/workflow.md`):

Instructs Conductor to load `writing-arch-specs` and align every track with `openspec/specs/<capability>/spec.md (REQ-XXXX)`.

**3. Post-Track Sync:** At Phase Checkpoint, run `/opsx:verify` and `/opsx:archive` to keep capability specs 100% current.

## The 7-Section Track `spec.md` Anatomy

Every `conductor/tracks/<track_id>/spec.md` MUST include:

1. **Overview** (2–3 sentence problem & outcome summary)
2. **Architecture & Component Topology** (Mermaid diagram)
3. **Functional Requirements** (`REQ-XXXX` + `WHEN/THEN`)
4. **Non-Functional Requirements** (latency, thread safety, > 80% cov)
5. **Acceptance Criteria** (1:1 mapping to `test_reqXXXX_*`)
6. **Out of Scope** (explicit guardrails against scope creep)
7. **Verification Commands** (`CI=true python3 -m unittest ...`)

## Why Dual-Engine SDD Beats Either Tool Alone

**Conductor alone** excels at single-track TDD execution and Git checkpoints, but archives track specs separately without merging them into a unified capability tree. **OpenSpec alone** maintains a clean capability spec tree (`openspec/specs/`), but lacks Conductor's `git notes` audit trail, `tech-stack.md` governance gates, and 4-phase `/conductor:revert`. **Together, they cover both planes.**

# Labs 01 & 02: Greenfield Spec Authoring & Brownfield Spec Recovery

Practicing forward specification from ambiguous proposals and reverse-engineering legacy code into contracts

## Lab 01: Greenfield Proposal → Specs, ADRs & Code

**System:** Per-Key Sliding-Window Rate Limiter (`ratelimiter.py`)

- **Starting Input:** Ambiguous product memo (`proposal.md`) missing cost bounds, retry-after math, and concurrency semantics.
- **What You Build:**
  - `openspec/specs/rate-limiter/spec.md` (REQ-0001..0006)
  - ADR-0001 (Storage Engine) & ADR-0002 (Sliding Log vs. Fixed)
  - 7-Section Conductor `spec.md`, `plan.md` & `ratelimiter.py`
- **Verify:** `./labs/lab_01_.../self_diagnose.sh` (6/6 tests PASS).

## Lab 02: Brownfield Legacy Code → Living Spec Recovery

**System:** Distributed Lease Lock Manager (`service/leasemanager.py`)

- **Starting Input:** Undocumented production Python service + unit tests with zero specification files.
- **What You Recover (/opsx:explore):**
  - REQ-0001..0006 in `openspec/specs/lease-manager/spec.md`
  - Strictly monotonic 64-bit `fencing_token` invariant (never resets across re-lease or TTL expiry)
  - ADR-0001-fencing-tokens.md (split-brain zombie protection)
- **Verify:** `./labs/lab_02_.../self_diagnose.sh` (6/6 tests PASS).

## Key Engineering Takeaway from Labs 01 & 02

Always require an injectable `clock: Callable[[], float] = time.monotonic` in your specification (REQ-0004) so time-window expiration, retry-after delays, and lease TTL takeovers can be unit-tested deterministically in `< 1ms` without `time.sleep()`.

# Labs 03 & 04: TDD State Machines, Drift Detection & 5-Step Refactoring

Translating formal specs into traceable TDD code and surgically remediating brownfield scorecard findings

## Lab 03: From Specs to Traceable TDD Code

**System:** Stateful Circuit Breaker (`circuitbreaker.py`)

- **Starting Input:** Approved `specs/spec.md` (REQ-0001..0007) and state machine diagram (`CLOSED` → `OPEN` → `HALF_OPEN`).
- **What You Build** (`/conductor:implement`):
  - Sequenced TDD `tasks.md` across 2 batches
  - `test_req0001_*` through `test_req0007_*` in `test_circuitbreaker.py`
  - Fast-fail `CircuitOpenError` & single-probe `HALF_OPEN` recovery
- **Verify:** `./labs/lab_03_.../self_diagnose.sh` (7/7 tests PASS).

## Lab 04: Spec Drift & 5-Step Scorecard Refactoring

**System:** Token Bucket Quota Allocator (`quota_allocator.py`)

- **The Drift Bug** (F-01 / REQ-0005): `_replenish()` overwrites `last_replenished = now` even when `added == 0`, starving token refill under frequent sub-second polling!
- **The 5-Step Brownfield Loop:**
  1. Audit Scorecard (F-01..F-04) → 2. Isolate F-01 in Track `spec.md` (Out of Scope: F-02..F-04) → 3. Write failing `test_req0005` → 4. Fix `_replenish()` → 5. Re-score.

## Why the “Out of Scope” Section in Lab 04 Is Critical

When given a brownfield file with 4 technical debt findings (F-01..F-04), an unconstrained agent attempts to fix all 4 at once and breaks existing callers. Placing F-02..F-04 in **Section 6: Out of Scope** forces surgical, 1-finding-per-track execution.

# Lab 05: SDD Code Review & Stacked PR Task Decomposition

Reviewing the specification diff first, blocking silent contract violations, and sizing PR batches

## Auditing `simulated_review/subpar_change.patch`

A contributor submits a “vibe-coded” patch to `base/kvcache.py`:

**Violation 1 (Silent Contract Break):** Adds `self._store.move_to_end(key)` inside `get()`, silently mutating **FIFO eviction (REQ-0002)** into **LRU eviction** without updating the spec!

**Violation 2 (Un-Specced Public API):** Adds `put_with_ttl()` with zero requirement in `openspec/specs/kv-cache/spec.md`.

**Violation 3 (Untested Edge Cases):** No `ttl_seconds <= 0` validation and no `test_req0006_*` unit test.

## Stacked PR Task Decomposition (`tasks.md`)

Instead of a monolithic PR, structure `tasks.md` into reviewable Stacked PR Batches (150–250 LOC each):

- **Batch 1 (PR 1/3):** Add **REQ-0006** to `spec.md` (keeping **REQ-0002** FIFO intact) + add injectable `clock` and `_CacheEntry`.
- **Batch 2 (PR 2/3):** Implement `put_with_ttl`, pre-eviction expired purging, and `test_req0001..0006`.
- **Archive PR N+1 (PR 3/3):** Run `/opsx:verify` & `/opsx:archive`.

## The Golden Rule of SDD Code Review

**Review the Specification Diff before reading a single line of Python/Go/TypeScript.** If a PR alters observable behavior without a matching **REQ-XXXX** delta—or if it contradicts an existing requirement—reject the PR at the contract layer.

# Lab 06: The 8-Section SPEC.md (SSOT) & Clean-Room Rebuild Test

Proving your specification is a true Single Source of Truth via isolated adversarial verification

## The Canonical 8-Section SPEC.md Blueprint

In `labs/lab_06_capstone_ssot_and_rebuild_test/SPEC.md`, the **CoursePulse Enrollment & Waitlist Service** is specified across 8 sections:

1. System Overview & Domain Intent
2. Architecture & Data Flow (Mermaid topology)
3. Data Models & Schemas (`EnrollmentResult`, `DropResult`, `CourseRoster`)
4. Interface & API Contracts (`create_course`, `enroll`, `drop`, `get_roster`)
5. Normative Workflows (`REQ-0001..0006`: FIFO waitlist promotion)
- 6–8. Repo Structure, Parameterized Config & 3-Phase Rebuild Recipe.

## The Adversarial Clean-Room Rebuild Protocol

How `labs/lab_06.../self_diagnose.sh` verifies Level-3 SSOT maturity:

**Step 1:** Create an empty isolated directory `mktemp -d` in `/tmp/`.

**Step 2:** Copy only `SPEC.md`, the regenerated `coursepulse.py`, and `adversarial_tests/test_rebuild_contract.py`.

**Step 3:** Lock `adversarial_tests/` read-only via `chmod -R a-w` so the implementer agent cannot weaken or delete test assertions.

**Step 4:** Execute `python3 -m unittest` — all 6 contract tests pass!

## Multi-Agent Adversarial Isolation (Preventing Sycophantic Tests)

In production SDD pipelines, **Agent A (Test Scaffold)** writes `adversarial_tests/` from `SPEC.md`, and **Agent B (Implementer)** writes `coursepulse.py` with filesystem write permissions restricted to `src/` only.

# 5 Mandatory Security Guardrails & Top Practitioner Traps

Avoiding silent scenario loss, parser drops, credential leaks, and mid-track tech-stack drift

## Top 5 Practitioner Failure Modes & Fixes

- 1. Partial MODIFIED Block:** Pasting only the new scenario under `## MODIFIED Requirements` overwrites the entire requirement on archive. *Fix: Copy the full requirement + all retained scenarios.*
- 2. 3-Hashtag ### Scenario: Drop:** OpenSpec's parser strictly requires 4 hashtags (#### Scenario:).
- 3. RENAMED + MODIFIED Collision:** `MODIFIED` must reference the `new (TO:)` header name.
- 4. Unapproved Stack Drift:** Halt task [~] and update `tech-stack.md` first.

## 5 Security Guardrails for SDD Repositories

- 1. Zero Secrets in Specs:** `openspec/` and `conductor/` are committed to Git—use `${API_KEY}` placeholders, never live tokens.
- 2. Sanitize Brownfield Extractions:** Strip internal hostnames, project IDs, and PII during `SPEC.md` extraction.
- 3. Enforce Plan Mode Path Sandboxing:** Keep `policies/conductor.toml` active so planning cannot mutate `src/`.
- 4. Lock Adversarial Tests Read-Only:** `chmod -R a-w adversarial_tests/`.
- 5. Gate CI on Strict Validation:** Run `./self_diagnose_all.sh` on every PR.

## Quick Diagnostic Command Cheat Sheet

- Validate all OpenSpec changes in CI: `openspec validate -all -strict -json`
- Audit Conductor tracks: `/conductor:status`
- Run Principal Engineer spec review: `/conductor:review`
- Logical rollback: `/conductor:revert`

# Start Practicing Now: Clone, Diagnose & Build

Everything in SDD-Crash-Course runs offline in < 1 second with Python 3.11+ Standard Library

## 2-Minute Terminal Quickstart

### 1. Clone the Open-Access Repository:

```
git clone https://github.com/adkaspar-google/sdd-course.git
cd sdd-course
```

### 2. Run the Master Diagnostic Suite (All 6 Labs):

```
./self_diagnose_all.sh
```

### 3. Install Conductor & OpenSpec in Your IDE/CLI:

```
gemini extensions install \
  https://github.com/gemini-cli-extensions/conductor
npm install -g @fission-ai/openspec@latest
```

## Included Course Resources & Codelabs

### • 16-Page Practical Field Guide PDF:

`Coursebook_SDD_Conductor_OpenSpec.pdf`

### • Dual-Engine Playbook:

`playbooks/DUAL_ENGINE_CONDUCTOR_OPENSPEC_PLAYBOOK.md`

### • Dedicated Spec Skill Template:

`.agents/skills/writing-arch-specs/SKILL.md`

### • Official Google Codelabs:

- `codelabs.developers.google.com/next26/advanced-planning`
- `codelabs.developers.google.com/conductor-plugin`

## Summary: Measure Twice, Specify Once, Code Deterministically

Whether you are building greenfield services (Lab 01 & 03), recovering specs and refactoring brownfield legacy systems (Lab 02 & 04), reviewing PRs (Lab 05), or running clean-room rebuilds (Lab 06), **the specification is your source of truth.**